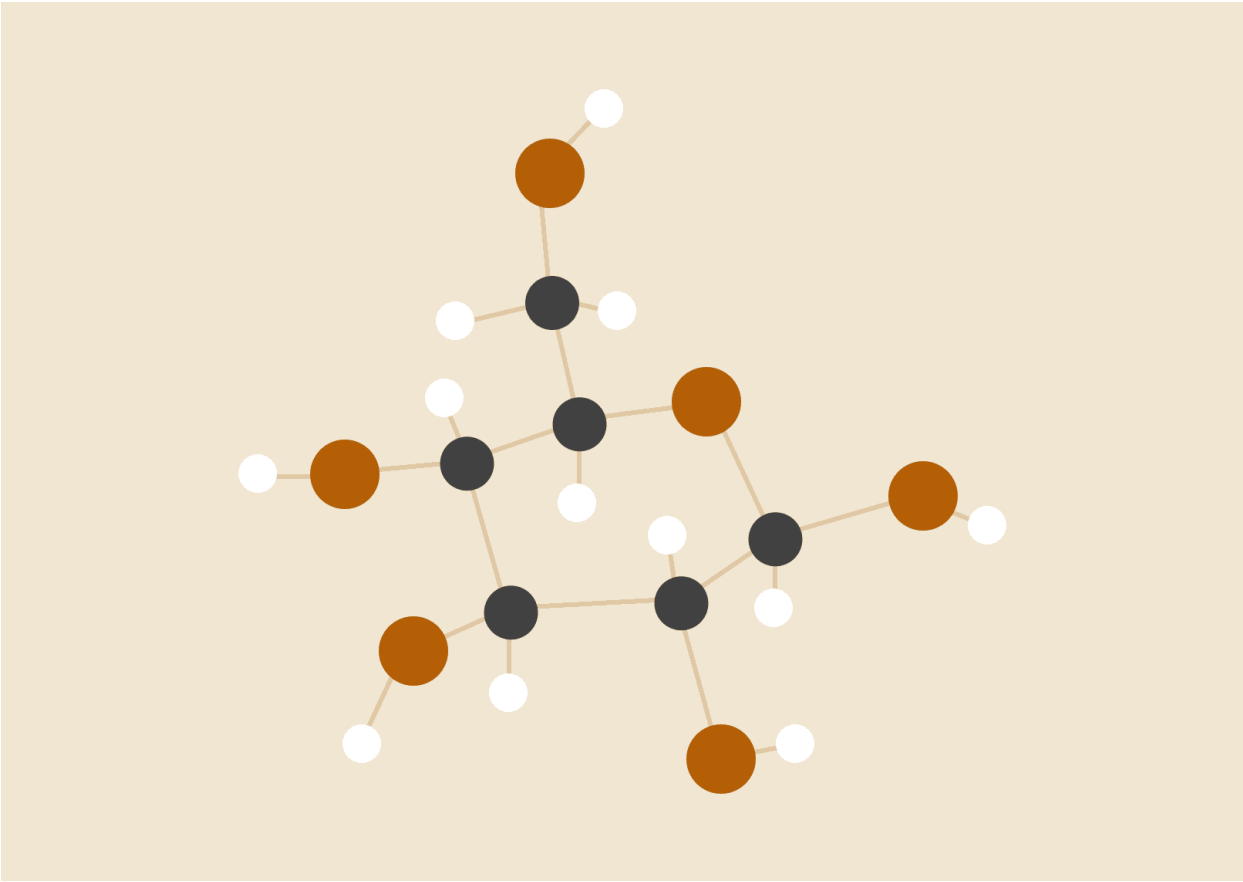
Project 3 Team Report

Team Rocket Escape



CS 2430

Brayden Bigelow - Communications Lead

Nathan Johnson - Verification Lead

Fall 2026

INTRODUCTION

In this project, our goal is to implement strategies for solving the knapsack problem. For our knapsack problem we must find the optimal payload for a rocket containing scientific experiments (The experiment list used in this report is shown in the table on the right). We want our payload to have the highest scientific value / rating possible without exceeding the rocket's weight limit. We must look at two different ways of solving this problem. The slow brute force way looks at every single combination of experiments to find the highest rated that is below the weight limit. It's not very efficient but it will find the best possible payload. The other, more efficient, way we must implement is greedy strategies, strategies which always choose the highest rated, lowest weight, or best rating to weight ratio. These strategies are a lot faster, but don't necessarily select the best payload possible.

Project Experiments	Weight (kg)	Rating
Cloud Patterns	36	5
Solar Flares	264	9
Solar Power	188	6
Binary Stars	203	8
Relativity	104	8
Seed Viability	7	4
Sun Spots	90	2
Mice Tumors	65	8
Microgravity Plant Growth	75	5
Micrometeorites	170	9
Cosmic Rays	80	7
Yeast Fermentation	27	4

APPROACH

We decided to implement these strategies in Java using the standard library (java.util) and wrote unit tests with JUnit 5. The project has 3 classes. The Experiment class represents our scientific experiments that have a name, weight, and rating and can compute its own rating to weight ratio. The Payload class holds a list of experiments and the total weight and rating of those experiments. The Strategies class contains the implementations of our strategies and a main method that prints sample output

GREEDY STRATEGIES

For all 3 greedy strategies, we used a greedy method that takes in a comparator, a max weight, and a list of experiments. All 3 greedy strategies pass their specific comparator, the user-defined max weight, and the constructed experiments list. The greedy method builds a sorted version of the provided experiments list based on the provided comparator. It then creates a payload and iterates through the sorted list adding each element until the provided weight limit is reached.

Highest Rating First:

Highest rating first provides the greedy method with a reversed Comparator.comparingInt, because comparingInt is ascending by default, that compares the experiment ratings and chooses the highest. It will then compare the weight if the ratings are equal and choose the lower weight. The resulting payload generated by the highest rating first greedy strategy is shown in the table on the right. It achieves a total rating of 45 with the given weight limit of 700.

Highest Rating First Payload	Weight	Rating
Micrometeorites	170	9
Solar Flares	264	9
Mice Tumors	65	8
Relativity	104	8
Cosmic Rays	80	7
Seed Viability	7	4
Total Weight And Rating:	**690**	**45**

Lightest First:

Lightest first provides the greedy method with a Comparator.comparingInt that compares the experiment weights choosing the lower one, it will then compare the experiments rating if the weights are equal using a reversed Comparator.comparingInt choosing the higher one. The resulting payload generated by the lightest first greedy strategy is shown in the table on the right. It achieves a total rating of 52 with the given weight limit of 700.

Lightest First And Highest Ratio Payload	Weight	Rating
Seed Viability	7	4
Yeast Fermentation	27	4
Cloud Patterns	36	5
Mice Tumors	65	8
Microgravity Plant Growth	75	5
Cosmic Rays	80	7
Sun Spots	90	2
Relativity	104	8
Micrometeorites	170	9
Total Weight And Rating:	**654**	**52**

Highest Ratio First:

Highest ratio first provides the greedy method with a reversed Comparator.comparingDouble that references the ratio method which calculates the rating divided by the weight and chooses the higher ratio. If the ratios are the same it then compares the weights using a comparingInt and chooses the lower weight. With a given weight limit of 700, it generates the same payload as the lightest first greedy strategy, with a total rating of 52.

BRUTE FORCE STRATEGY

For the brute force strategy, since each experiment is either in or out, we represent each payload as a n-bit number where n is the number of experiments in the experiment list. For a list of 12 like our example case, that would represent a number from 0 to 4095. The program would then look at every number from 0 to 4095 and use a bit mask to check each bit position of that number, and if the bit is 1 at that position it will add the corresponding experiment to the payload. Once the experiments have been added, it will then check if payload is within the weight limit, and if it is, it will add the payload to a list of valid payloads. Once all possible payloads have been looked through it will return the list of valid payloads that can be sorted through elsewhere.

Brute Force #1 Solution	Weight	Rating
Cloud Patterns	36	5
Binary Stars	203	8
Relativity	104	8
Seed Viability	7	4
Mice Tumors	65	8
Micrometeorites	170	9
Cosmic Rays	80	7
Yeast Fermentation	27	4
Total Weight And Rating:	**692**	**53**

Brute Force #2 Solution (Same as Lightest First and Ratio First)	Weight	Rating
Seed Viability	7	4
Yeast Fermentation	27	4
Cloud Patterns	36	5
Mice Tumors	65	8
Microgravity Plant Growth	75	5
Cosmic Rays	80	7
Sun Spots	90	2
Relativity	104	8
Micrometeorites	170	9
Total Weight And Rating:	**654**	**52**

Brute Force #3 Solution	Weight	Rating
Seed Viability	7	4
Yeast Fermentation	27	4
Cloud Patterns	36	5
Mice Tumors	65	8
Solar Power	188	6
Cosmic Rays	80	7
Relativity	104	8
Micrometeorites	170	9
Total Weight And Rating:	**677**	**51**

VERIFICATION

We used JUnit tests in order to verify our implementation and check edge cases. We first build a small four experiment test list (see right table) that allows us to easily manually find and set the expected output for each strategy.

Test Experiment List	Weight	Rating
Foo	3	1
Bar	4	2
Baz	5	3
Bass	10	4

RESULTS

Strategy	Experiments Count	Total Weight (kg)	Total Rating	Percent Of Optimal
Highest Rating First	6	690	45	84.9%
Lightest First	9	654	52	98.1%
Best Ratio First	9	654	52	98.1%
Brute Force (optimal)	8	692	53	100%

None of the 3 greedy strategies found the optimal payload, but the lightest first and best ratio first both found the 2nd best payload with a rating of 52 (1 point below optimal). Highest rating first performed worst with a rating of 45 (8 points below optimal). The brute force search was the only strategy that found the optimal payload with a rating of 53.

ANALYSIS

The brute force strategy is the only strategy here that is guaranteed to find the optimal answer, but it can only do so by checking every single payload, which is very slow and runs in O(n*2^n) time. The greedy strategies are much faster and all run in O(n log n) time but with the downside of not always finding the exact optimal solution. Highest rating first fails to find the optimal solution because it ignores the weight cost of each experiment, the experiments with the highest rating, Solar Flares and Micrometeorites, each have a rating of 9, but take up 60% of the available weight with very bad rating to weight ratios of 0.034 and 0.053 rating per kg. Lightest first fails

because it ignores value. It performed well with the given experiment list because the lightest experiments happened to have very good ratings to weight, like seed viability and yeast fermentation's ratios of 0.57 and 0.14 respectively.

MATERIALS

1. Lorem ipsum dolor sit amet
2. Consectetuer adipiscing elit
3. Sed diam nonummy nibh euismod